Numerical Analysis: Final exam (**70 marks**)

Urbain Vaes

Fall $(11111100101)_2$

You are not required to complete every question. Although the total marks on the exam sum to 70 (not including bonus points), your final grade will be calculated out of 60.

Academic integrity pledge: ☐ I certify that I will not give or receive any unauthorized help on this exam, and that all work will be my own. (Tick ✓ or copy the sentence on your answer sheet).

Question 1 (Floating point arithmetic, **10 marks**). True or false?

1. Let $(\bullet)_2$ denote binary representation. Then $(1000000000)_2 \times (0.001)_2 = (1000000)_2$.
2. It holds that $(0.111111)_2 + (0.000001)_2 = (1.001)_2$.
3. It holds that $(0.\overline{1100})_2 = \frac{4}{5}$.
4. Let ε_{32} denote the machine epsilon for the `Float32` format, i.e. `eps(Float32)`. Then the number 3 is representable exactly in this format, and the next representable number in the `Float32` format is $3 + 2\varepsilon_{32}$.
5. In Julia, `Float64(3.25) == Float32(3.25)` evaluates to `true`.
6. The spacing (in absolute value) between successive half-precision (`Float16`) floating point numbers is always greater than or equal to `eps(Float16)`.
7. In Julia, define `f(x) = x == Inf ? - x : f(-2x)`. Then `f(5.0)` evaluates to `Inf`.
8. There exists $n \in \mathbf{N}$ such that the machine epsilon for the `Float64` format equals 2^{-n}. In other words, the machine epsilon is an exact negative power of two.
9. In Julia, it holds that `eps()/3 + eps()/3 + 1 > 1` evaluates to `true`.
10. In Julia, the expression `1 + 1024eps() == 2` evaluates to `true`.
11. **Bonus.** Does the Julia expression `log2(2 + eps()) == 2` evaluate to `true` or `false`.

    Explain briefly:

12. **Bonus.** Does the Julia expression `sqrt(4 + 5eps()) == 2` evaluate to `true` or `false`?

    Explain briefly:

Question 2 (Interpolation, **10 marks**)**.** The first part of this question is an open-ended question, while the second part is a true/false.

1. (**5 marks**, open-ended question) Describe the Gregory–Newton method to calculate the interpolation polynomial in the case of equally spaced interpolation nodes at $\{0, 1, 2, \cdots, n\}$ and corresponding values $\{f(0), f(1), f(2), \cdots, f(n)\}$. In particular, address the following points:

   - Write down the expression for the interpolating polynomial;
   - Discuss the advantages and disadvantages of this interpolation method compared to Lagrange interpolation;
   - Illustrate the method on a concrete example of your choice.

2. (**5 marks**) Are the following assertions true or false?

   - Let $f(x) = 2x^4 + x^2 - 3x - 1$, and let $f_3 \in \mathcal{P}_3$ denote the polynomial interpolant of f at four points $x_0 < x_1 < x_2 < x_3$. Then

   $$f(x) - f_3(x) = \frac{2}{4!}(x - x_0)(x - x_1)(x - x_2)(x - x_3)\,.$$

   - Let $f\colon \mathbf{R} \to \mathbf{R}$ be the Runge function defined by $f(x) = (1 + 25x^2)^{-1}$, and let $f_n \in \mathcal{P}_n$ denote the polynomial interpolating f at $n+1$ equally spaced points in the interval $[-1, 1]$. Then, it holds that

   $$\lim_{n\to\infty} \left(\max_{x\in[-1,1]} \left| f(x) - f_n(x) \right| \right) = 0\,.$$

   - Let $f(x) = \sin(\pi x) + \mathrm{e}^x$, and let f_n denote the piecewise linear interpolation of f at $n+1$ equally spaced points in the interval $[-1, 1]$. Then, it holds that

   $$\lim_{n\to\infty} \left| \int_{-1}^{1} f(x) - f_n(x)\,\mathrm{d}x \right| = 0\,.$$

   - Consider the function $\boldsymbol{F}\colon \mathbf{R}^{n+1} \to \mathbf{R}$ defined by

   $$F(\boldsymbol{x}) \coloneqq \max_{z\in[-1,1]} \left| (z - x_0)(z - x_1)\cdots(z - x_n) \right|, \qquad \boldsymbol{x} = (x_0, x_1, \ldots, x_n) \in \mathbf{R}^{n+1}\,.$$

   Then $\boldsymbol{F}$ is *maximized (as large as possible)* when the points $x_0, x_1, \ldots, x_n$ are chosen as the roots of the Chebychev polynomial of degree $n + 1$.

   - In the following code, `p` is the interpolating polynomial through the data in `x` and `y`. Here, for an array `y` of size n, the command `diff(y)` returns the array of size $n - 1$ whose i-th entry is equal to `y[i+1] - y[i]`.

```
x = [0, 1, 2, 3]
y = [1, 2, 1, 2]

function p(x)
    return (y[1]
            + diff(y)[1] * x
            + 1/2 * diff(diff(y))[1] * x^2
            + 1/6 * diff(diff(diff(y)))[1] * x^3)
end
```

   - (**Bonus, 1 mark**) Let $f(n) = 2^n$. Then for all $n \in \mathbf{N}$, it holds that

   $$f(n) = \sum_{i=0}^{+\infty} \frac{\Delta^i f(0)}{i!} n^{\underline{i}},$$

   where Δ denotes the forward difference operator, meaning that $\Delta f(n) = f(n+1) - f(n)$, Justify your answer.

Question 3 (Numerical interpolation, **10 marks**)**.** We wish to derive and implement Boole's rule for numerical integration. Boole's rule is a Newton–Cotes quadrature rule based on five equally spaced points in the interval $[-1, 1]$.

$$I[u] := \int_{-1}^{1} u(x)\,\mathrm{d}x \approx w_0 u(-1) + w_1 u\left(-\frac{1}{2}\right) + w_2 u(0) + w_3 u\left(\frac{1}{2}\right) + w_4 u(1) =: \widehat{I}_5[u]. \tag{1}$$

1. (**5 marks**) Calculate the weights w_0, w_1, w_2, w_3, w_4 to ensure that the quadrature rule $\widehat{I}_5$ is exact for all polynomials in $\mathcal{P}_d$, where d is as large as possible. Include the details of your calculation.

   ***Hint**: It is useful to require that the quadrature rule is exact for the Lagrange basis polynomials.*

   ***Hint**: By symmetry, you can expect that $w_0 = w_4$ and $w_1 = w_3$.*

   Your answer:

2. (**1 mark**) What is the degree of precision of Boole's rule (1)?

   Your answer:

3. (**4 marks**) Write a function `boole(u, a, b)` that returns, using Boole's rule appropriately adapted to the interval $[a, b]$, an approximation of the integral

$$\int_a^b u(x)\,\mathrm{d}x\,. \tag{2}$$

```
function boole(u, a, b)
    # Write your code here




end
```

4. (**Bonus**, **2 marks**) Write a function `composite_boole(u, a, b, N)` that returns an approximation of the integral (2), this time using a composite version of Boole's rule. More precisely, the approximation should be obtained by partitioning the integration interval $[a, b]$ into N cells, and applying Boole's rule within each cell. Your function should call the function `boole` you wrote in the previous item.

```
function composite_boole(u, a, b, N)
    # Write your code here




end
```

Question 4 (Iterative method for linear systems, **10 marks**)**.** Assume that $\mathsf{A} \in \mathbf{R}^{n\times n}$ is a nonsingular matrix and that $\boldsymbol{b} \in \mathbf{R}^n$. We wish to solve the linear system

$$\mathsf{A}\boldsymbol{x} = \boldsymbol{b} \tag{3}$$

using an iterative method where each iteration is of the form

$$\mathsf{M}\boldsymbol{x}_{k+1} = \mathsf{N}\boldsymbol{x}_k + \boldsymbol{b}. \tag{4}$$

Here $\mathsf{A} = \mathsf{M} - \mathsf{N}$ is a splitting of A such that M is nonsingular, and $\boldsymbol{x}_k \in \mathbf{R}^n$ denotes the k-th iterate of the numerical scheme.

1. (**3 marks**) Let $\boldsymbol{e}_k \coloneqq \boldsymbol{x}_k - \boldsymbol{x}_*$, where $\boldsymbol{x}_*$ is the exact solution to (3). Prove that the error satisfies the recurrence relation
$$\boldsymbol{e}_{k+1} = \mathsf{M}^{-1}\mathsf{N}\boldsymbol{e}_k.$$

2. (**3 marks**) Let $L = \|\mathsf{M}^{-1}\mathsf{N}\|_1$. Prove that
$$\forall k \in \mathbf{N}, \qquad \|\boldsymbol{e}_k\|_1 \leqslant L^k \|\boldsymbol{e}_0\|_1. \tag{5}$$

3. (**1 mark**) In general, is the condition $\|\mathsf{M}^{-1}\mathsf{N}\|_1 < 1$ sufficient for convergence (that it, does it ensure that $\boldsymbol{x}_k \to \boldsymbol{x}_*$ for every $\boldsymbol{x}_0$)? Justify briefly your answer.

4. (**1 mark**) In general, is the condition $\|\mathsf{M}^{-1}\mathsf{N}\|_1 < 1$ necessary for convergence? Justify briefly your answer.

5. (**2 marks**) Let $\boldsymbol{r}_k \coloneqq \mathsf{A}\boldsymbol{x}_k - \boldsymbol{b}$ denote the residual at iteration k. Prove that

$$\boldsymbol{r}_{k+1} = \mathsf{N}\mathsf{M}^{-1}\boldsymbol{r}_k \,.$$

💻 **Implementation exercise 1** (Linear systems, **10 marks**)**.** Assume that $\mathsf{A} \in \mathbf{R}^{n \times n}$ is an *upper triangular* matrix with nonzero diagonal entries (hence invertible) and that $\boldsymbol{b} \in \mathbf{R}^n$. We wish to solve the linear system

$$\mathsf{A}\boldsymbol{x} = \boldsymbol{b}. \tag{6}$$

- (**3 marks**) Solve the linear system (6) by backward substitution, using pen and paper, in the specific case where A and $\boldsymbol{b}$ are given by

```
A = [1.0 2.0 3.0; 0.0 4.0 5.0; 0.0 0.0 6.0]
b = [6.0; 9.0; 6.0]
```

- (**4 marks**) Suppose that A is upper triangular. Write a function `backward_subs(A, b)` implementing the backward substitution algorithm for solving (6) in this case.

```
function backward_subs(A, b)
    n = size(A, 1)
    x = zeros(n)









    return x
end
```

- (**3 marks**) How many floating point operations (additions, subtractions, multiplications, divisions) does your implementation of backward substitution perform in order to solve a linear system of size n? Include the details of your calculation.

- (**Bonus, 2 marks**) Assume now that A is a banded upper triangular matrix with bandwidth 1, which means that nonzero entries are only on the main diagonal and first superdiagonal. Write a function `banded_backward_subs(A, b)` that implements an efficient version of backward substitution for this case.

```
function banded_backward_subs(A, b)
    n = size(A, 1)
    x = zeros(n)









    return x
end
```

Question 5 (Iterative method for nonlinear equations, **10 marks**)**.** Consider the problem of finding a root of a nonlinear function $f\colon \mathbf{R} \to \mathbf{R}$.

- (**2 marks**) Assume that a numerical scheme of the form

$$x_{k+1} = F(x_k) \tag{7}$$

is employed to find a root of f, where $F\colon \mathbf{R} \to \mathbf{R}$ is a given function. Assume additionally that F is globally Lipschitz continuous with Lipschitz constant $L < 1$, and that there exists a fixed point x_* of F. First, prove that the fixed point x_* is unique.

- (**3 marks**) Under the same assumptions as in the previous item, prove that the sequence of iterates $(x_k)_{k\in\mathbf{N}}$ converges to x_* for any initial guess x_0.

- (**3 marks**) We wish to compute $x_* = \sqrt[4]{2}$. Propose an iterative scheme of the form (7) to approximate this value. Write down the expression of the function F you propose. Is the convergence of your scheme superlinear? Justify your answer by examining the value of $F'(x_*)$.

- (**2 marks**) Illustrate graphically the convergence of your scheme from the previous item, for the specific case where the initial guess is $x_0 = 1$.

💻 **Implementation exercise 2** (Iterative methods for nonlinear equations, **10 marks**)**.** We wish to calculate the point $(x_*, y_*) \in \mathbf{R}^2$ on the parabolic curve $y = x^2$ that is closest to the point $(3, 1)$.

- (**2 marks**) Find a function $f\colon \mathbf{R} \to \mathbf{R}$ such that the root of f corresponds to the x-coordinate of this point:

  $$f(x) =$$

- (**2 marks**) Let ε denote the dual unit satisfying $\varepsilon^2 = 0$. Calculate $f(x + \varepsilon)$ for $x \in \mathbf{R}$, and deduce from the result an expression for $f'(x)$.

  $$f(x + \varepsilon) =$$

  $$f'(x) =$$

- (**4 marks**) Implement the Newton–Raphson method to find an approximation of the root of a general differentiable function $g\colon \mathbf{R} \to \mathbf{R}$, starting from the initial guess x_0. Use an appropriate stopping criterion, and use automatic differentiation to compute the derivative g'. Your function should return only the final approximation of the root.

  Hint: Recall that `ForwardDiff.derivative(g, x)` computes the derivative of g at x.

```
using ForwardDiff
function newton_raphson(f, x0; tol = 1e-12, maxiter = 10_000)
    # Your code here









end
```

- (**2 marks**) Use your implementation of the Newton–Raphson method (i.e., call the function with appropriate arguments) to find an approximation of the point (x_*, y_*) introduced above, and print the result.

```
# Your code here
```

- (**Bonus, 2 marks**) The secant method is an iterative method for finding a root of a function $f\colon \mathbf{R} \to \mathbf{R}$, where each iteration depends on the two previous iterates, according to the formula

$$x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}\,.$$

Implement the secant method to find an approximation of the root of f, starting from the initial guesses x_0 and x_1.

```
function secant(f, x0, x1; tol = 1e-12, maxiter = 10_000)
    # Your code here









end
```